CSC317 Computer Graphics Tutorial 7

November 06, 2025

Assignment 6: Reminders

- Assignment 6 due tomorrow: Nov 7, 2025 before 11:59 pm
- Questions and Assistance
  - “Issues” page on github for Assignment 6
  - Email the TAs: csc317tas@cs.toronto.edu

Assignment 7

- Assignment 7 Due Date: November 14, 2025 before 11:59 pm
- Questions and Assistance
  - “Issues” page on github for Assignment 7
  - Email the TAs: csc317tas@cs.toronto.edu
- Office Hour via Zoom: November 14, 1:00 pm - 3:00 pm

Task 1: src/euler_angles_to_transform.cpp

- Goal: Transform Eulerian angles to a 4x4 rotation matrix
- Input: Three Euler angles rotating along the x-, z-, and x-axes.
- Output: A 4x4 transformation matrix.

Task 1: src/euler_angles_to_transform.cpp

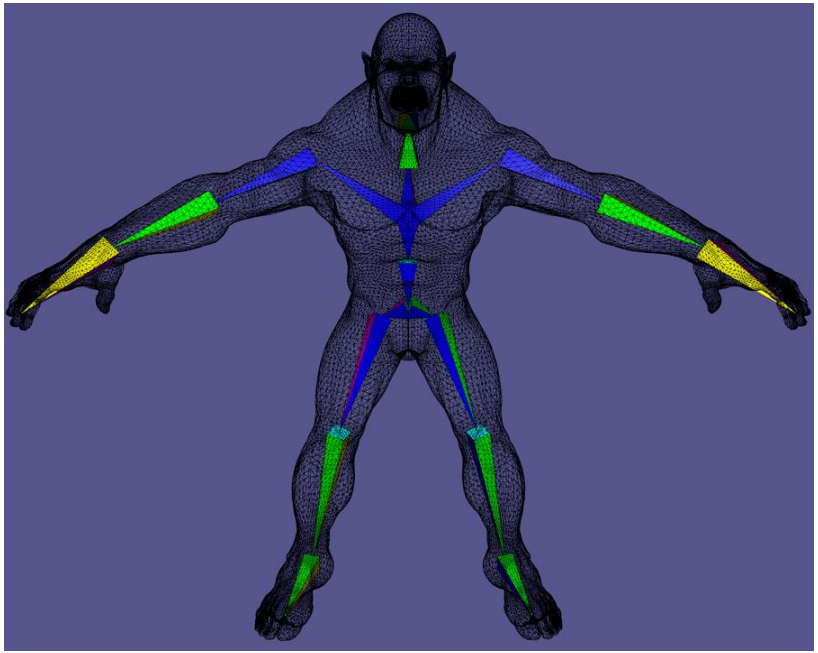
- General idea: We can apply rotation along the x-, z-, and x-axes sequentially.
- Using **Eigen::AngleAxis** to represent a 3D rotation:
  - Inputs: (angle, axis).
- Using Eigen::Vector3d::UnitX/Y/Z() to represent the X/Y/Z axis respectively.
- Note: the angle should be in in radian and the axis must be normalized.

Task 2: src/forward_kinematics.cpp

- Goal: Compute the transformation of each skeleton joint
- Input: A transformation hierarchy of different bones.
- Output: An affine transformation vector for each bone.

Task 2: src/forward_kinematics.cpp

- General idea: We can accumulate the transformation matrix from the root to the current bone:
  - Root -> Parents -> Current bone.



Task 2: src/forward_kinematics.cpp

- General idea: We can accumulate the transformation matrix from the root to the current joint: Root -> Parents -> Current joint.
- Steps:
  - Iterate over each bone and extract its rotation angles & parent index;
  - Translate the Euler angles to a rotation matrix for each bone;
  - Compute transformation matrices recursively from root to each bone.

Task 3: src/transformed_tips.cpp

- Goal: Compute positions of specified bone "tips" given a skeleton.
- Input:
  - List of bones with relative transformations
  - List of indices of skeleton endpoints to compute
- Output: Transformed tip positions.

Task 3: src/transformed_tips.cpp

- Steps:
  - Compute each tip position in the canonical bone;
  - Compute each tip position in the rest bone;
  - Compute each tip position in the pose bone with transformations from forward kinematics.

[Summarized from https://github.com/ohnooj/computer-graphics-kinematics/tree/master?tab=readme-ov-file]

Task 4: src/catmull_rom_interpolation.cpp

- Goal: Interpolate at time t using a Catmull-Rom spline.
- Input:
  - Keyframe information: list of pair (time, Euler angles)
  - Query time
- Output: Interpolated angles at time t.

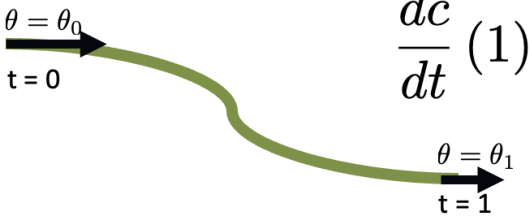
Task 4: src/catmull_rom_interpolation.cpp

Catmull-Rom Spline

$$\mathbf{c}(t) = at^3 + bt^2 + ct + d$$
$$\mathbf{c}'(t) = 3at^2 + 2bt + c$$

A **cubic** curve created by specifying the end points and the tangents of the curve.

$$c(0) = d$$
$$c(1) = a + b + c + d$$
$$\frac{dc}{dt}(0) = 1c$$
$$\frac{dc}{dt}(1) = 3a + 2b + 1c$$



General idea:

We need to compute the values and corresponding tangents of two nearby keyframes.

Task 4: src/catmull_rom_interpolation.cpp

- Steps:
  - Search two nearby keyframes given time t;
  - Compute the values and tangents at time t_0 and t_1 and then coefficients of c(t);
  - Given t, compute parametric distance between two keyframes as

$$t = (t' - t_0)/(t_1 - t_0)$$

  - Obtain final interpolated values by substituting t into c(t).

Task 5: src/linear_blend_skinning.cpp

- Goal: Compute deformed mesh vertex positions with the linear blend skinning deformation.
- Input:
  - Mesh vertex positions in rest pose
  - List of skeleton bones
  - List of affine transformations
  - List of weights corresponding affine transformations
- Output: List of deformed mesh vertex positions.

Task 5: src/linear_blend_skinning.cpp

- Steps:
  - Iterate over each mesh vertex;
  - The position of each vertex is a linear combination of the transformation of each bone as

$$\boldsymbol{v}_j = \sum_{i=1}^{nB} w_{ij} \boldsymbol{T}_i \hat{\boldsymbol{v}}_j$$

where nB is the number of bones, w_{ij} is the weight of bone i on vertex j, $\boldsymbol{T}_i$ is the transformation of bone i, $\boldsymbol{v}_j$ is the position of vertex j on mesh, and $\hat{\boldsymbol{v}}_j$ is the position of vertex j on the rest pose mesh.

Task 6: src/copy_skeleton_at.cpp

- Goal: Copy data and return a new Skeleton instance.
- Iterate over each bone in the skeleton and do the copying.

Task 7: src/kinematics_jacobian.cpp

- Goal: Find the Jacobian matrix of the forward kinematics function.
- Input:
  - Skeleton information
  - Index list of skeleton endpoints to consider
- Output: A Jacobian matrix.

Task 7: src/kinematics_jacobian.cpp

- Steps:
  - Iterate over each bone of the skeleton
  - Then iterate over each angle of each bone (A bone has three angles)
  - Apply small changes to the current angle
  - Extract transformed positions of tips to consider
  - Compute Jacobian element with $\mathbf{J}_{i,j} \approx \frac{\mathbf{x}_i(\mathbf{a} + h\delta_j)}{h}.$ $h = 10^{-7}$

Task 8: src/end_effectors_objective_and_gradient.cpp

- Goal: Provide functions to computes the least-squares objective value, least-squares objective gradient and project a given set of Euler angles.
- Input:
  - List of bones
  - List of bone indices
  - List of end-effector positions
- Output: Three function handles.

Task 8: src/end_effectors_objective_and_gradient.cpp

- Steps to compute least-squares objective value:
  - Compute the transformed tip positions with the skeleton and bone indices
  - Compute squared 2-norm between transformed tip positions and end_effector positions with the squaredNorm() function

Task 8: src/end_effectors_objective_and_gradient.cpp

- Steps to compute least-squares objective gradient:
  - Compute the transformed tip positions with the skeleton and bone indices
  - Compute the Jacobian matrix and the difference as

$$\frac{df(\theta)}{d\theta} = \frac{d||\boldsymbol{x}(\theta) - \boldsymbol{q}||_2^2}{d\theta}$$
$$= 2\frac{d\boldsymbol{x}(\theta)}{d\theta}(\boldsymbol{x}(\theta) - \boldsymbol{q})$$

Task 8: src/end_effectors_objective_and_gradient.cpp

- Steps to project a given set of Euler angles:
  - Iterate over each bone of the skeleton
  - Clip each bone’s angles to an admissible range

Task 9: src/line_search.cpp

- "Line search" means search for a good step fraction α along the line search direction $\nabla_z f(z)$, such that $f(z - \alpha \nabla_z f(z))$ is smaller than $f(z)$.
- Steps:
  - Compute the initial function value as f(z) and initialize step size alpha;
  - Update the position as zj = z + alpha * dz, dz is the change of z;
  - Project zj and compute the function value f(zj);
  - Check whether f(zj) – f(z) is smaller than a threshold; $E(\text{proj}(\mathbf{a} + \sigma \Delta \mathbf{a})) < E(\mathbf{a})$
  - If not, reduce alpha by half and iterate until the maximum iteration number;
  - Return alpha.

Task 10: src/projected_gradient_descent.cpp

- Goal: Make results of each gradient descent step be in an admissible range
- Steps:
  - For every step of gradient descent, compute the gradient direction at z;
  - Then find the step size alpha with line search;
  - Update z and project z back to the constraint set after each update.